

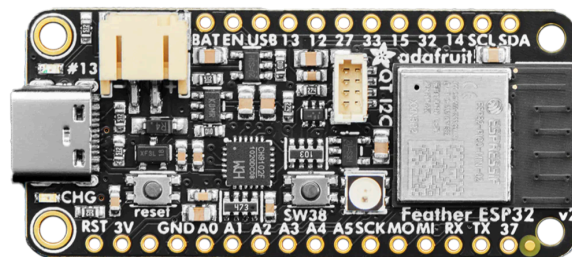
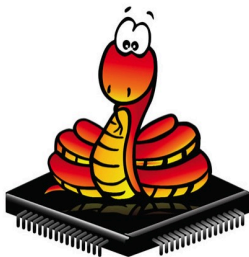
Artificial Intelligence of Things - Intelligent and Connected Systems

EECS E4764
Columbia University

Prof. Xiaofan Jiang

Fall 2026

**Lab 4 - Bus Communication: SPI
and Interfacing with the World**



Introduction

An important concept behind many innovations and advancements is to build upon or improve what already exists. In other words, it is in your best interest to use existing systems that accomplish what you plan to build upon or use as a foundation for your own work. This not only saves you time because you can spend it elsewhere working on your own key contributions rather than reinventing something that has already been done, but it can also make your own products or innovations easier to use or interface with since they build upon existing technologies. This is especially true for IoT systems and devices; one of the primary aims for IoT products is not only to allow for a network of communication between physical devices and systems, but to also do so in a convenient manner by utilizing existing and flexible infrastructures.

In this lab, we will use APIs to add some nifty features to our wearable. An API (application programming interface) makes the primary functions and commands available to the user or developer, so that they can utilize the existing technology or functionalities that the API provides. Just as a computer screen shows the user what is available or the functionalities on that computer, an API shows the developer what features of the technology are available to use. While a user can communicate actions and commands to a computer through peripherals, like a mouse and keyboard, a developer can communicate with many APIs through HTTP, an application layer communication protocol, or through the RESTful API, which uses HTTP as its basis.

For our embedded system, we will be using the IP Geolocation API, and the OpenWeatherMap API to track the weather in our area and the ntfy.sh API to send push notifications with our wearable.

Part 1: Interfacing with Accelerometer

Task 1

Interface with the accelerometer using SPI. Read the acceleration on three axes continuously and print them out. Tilt the accelerometer to confirm that the values make sense.

Consult the datasheet for its SPI timing diagram and registers you need to use.

<https://www.sparkfun.com/datasheets/Sensors/Accelerometer/ADXL345.pdf>

Task 2

Use the accelerometer readings and implement text scrolling; if you tilt the accelerometer far enough in one of the four principal directions (up, down, left, right), the text on the screen should begin scrolling in the same direction. For example, if you tilt the accelerometer to the right, the text on the OLED screen scrolls to the right until it goes off screen before it reappears on the left side of the screen.

Checkpoint 1

On top of the combined system from the last lab, implement text scrolling based on accelerometer readings.

Part 2: Geolocation and Weather

The objective of this part is to display weather information onto our smartwatch. To achieve this, we will first use a geolocation API to figure out where our current location is, then send our location (latitude and longitude) into a weather API to find out what's the weather in the area.

You will find the **urequests** and **ujson** libraries useful. Also, because they are MicroPython ports for the regular Python [requests](#) and [json](#) library, their usages are roughly the same. Thus, it may be easier for you to test your API codes on your computer with the requests and json libraries, then move the code onto your ESP.

You can also download [Postman](#) to help you test different APIs. It can also generate some code for you!

Task 1

Connect your ESP32 to the Internet. You can follow the documentation here: <https://docs.micropython.org/en/latest/esp32/quickref.html#networking>

Task 2

Retrieve the latitude and longitude of the current location. Display the latitude and longitude on the OLED screen. A simple API to use is: <http://ip-api.com/json>. More information about this API can be found in their documentation: <https://ip-api.com/docs/api:json>

(You can also use other api as long as the general function is the same)

Checkpoint 2

Interface with a Geolocation API and display your coordinates on the OLED display.

Task 3

Feed the coordinates from the geolocation API into the OpenWeatherMap API to receive weather data for your location. Display the current weather on the OLED display. You should at least display the temperature and description (e.g. mostly cloudy). More information about the OpenWeather API can be found at <http://openweathermap.org/api> and <http://openweathermap.org/current>. (You can also use other api as long as the general function is the same)

Checkpoint 3

Feed your coordinates into an API to receive information about the weather in your area; display the results on your OLED display.

Part 3: Push Notifications

Task

Interface with the ntfy.sh API to send push notifications from your ESP32.

The push notifications in Ntfy.sh (and many other frameworks) work in a pub/sub (publish/subscribe) fashion. Instead of sending the message directly from A to B, there is a **topic** (channel) B **subscribes** to. A **publishes** the message to the topic, and all subscribers to the channel receive the message.

To subscribe and read the message on a specific channel, go to the following URL in your browser. It's also available as a mobile phone app.

https://ntfy.sh/<channel_name>.

It takes a simple API call to publish a message to a channel. Take a look at the documentation (<https://docs.ntfy.sh/>) to see how it works. Remember to use a unique **topic name** (such as your UNI), to avoid the message colliding with other people's.

Checkpoint 4

Send a push notification from the ESP32. An example notification could be your location coordinates with the weather in that location. Show the message by subscribing to the topic in the browser or your phone.

Lab 4 Checkpoints:

1. On top of the combined system from the last lab, implement text scrolling based on accelerometer readings.
2. Interface with a Geolocation API and display your coordinates on the OLED display.
3. Feed your coordinates into an API to receive information about the weather in your area; display the results on your OLED display.
4. Send a push notification from the ESP32. An example notification could be your location coordinates with the weather in that location. Show the message by subscribing to the topic in the browser or your phone.

Submission:

1. Write a brief description "report" about your understanding of SPI communication used by the accelerometer (checkpoint 1) for this lab.
 - How to wire up the accelerometer to the board
 - How the read/write is performed
2. Submit the file for each checkpoint and writeup in 1) onto courseworks → Assignments → Lab4

The submitted files should be named like

```
labx_{member1_uni}_{member2_uni}_{member3_uni}_{member4_uni}_check{X}.py
```

There should be 5 files for this lab.

e.g.

- Lab4_..._writeup.txt
- Lab4_..._check1.py
- Lab4_..._check2.py
- Lab4_..._check3.py
- Lab4_..._check4.py